FitKV

Training free KV cache compression that sets a **per input** budget.

FitKV reads how much the final prompt queries disagree about which parts of the context to attend to, in the late layers of the model. Inputs whose queries agree keep almost their full cache; inputs whose queries disagree shrink toward a minimum budget that you set. Blocks are then selected per layer with the same attention statistics and a value term.

How it works

For each late layer (the last third of the layers):

1. The last `w` prompt queries attend to the context, pooled into blocks of `P` tokens.
2. Each block gets a mean `mu` and a standard deviation `sigma` over the queries.
3. The layer's disagreement is `D = mean(sigma) / mean(mu)`.

The routing scalar is `lambda = min(1, mean(D) / 2)`, and the budget of the input is

```
B_x = floor(n - lambda * (n - B))
```

where `n` is the prompt length and `B` is the minimum budget. Every layer then keeps the `floor(B_x / P)` highest scoring blocks, always including the observation window and the last two blocks.

The block score is `mu + lambda_l * sigma + eta * g`, where `g` is a value term that measures how far a block's mean value lies from the window's attention output.

Baselines

The repository includes the baselines compared with FitKV in the paper. They share the same block size, observation window, always kept blocks and budget floor, so the differences come from the selection rule only.

Method	Name in `apply_method`	Selection rule
Full cache	`full`	Keeps every token
FitKV	`fitkv`	Adaptive budget and block selection (this method)
FitKV, fixed budget	`fitkv_fixed`	FitKV block selection with `lambda = 1`
SnapKV	`snapkv`	Attention of the final window, pooled over the window
H2O	`h2o`	Attention accumulated over all prompt queries; sink blocks kept
Quest	`quest`	Upper bound of query and key dot products per block
PyramidKV	`pyramidkv`	SnapKV scores with a linear budget schedule across layers
Ada-KV	`adakv`	Per head SnapKV scores with global competition and a per head floor

```python
from fitkv.compare import apply_method
budget = apply_method("snapkv", keys, values, queries, minimum=128)
```

Notes on the baselines:

- **Quest** is applied once after the prompt, with its bound computed from the last query. The original Quest design selects pages at every decoding step, so these numbers measure Quest as a one shot eviction method.
- **PyramidKV** uses a per layer budget that decreases with depth and averages to the minimum `B`. Its per layer cache therefore differs from the other methods, while the mean is the same.
- **Ada-KV** is a block level adaptation of AdaKV. The original method works on tokens within each head, so this version is an approximation at block granularity.

Install

```
pip install -r requirements.txt
```

Use

```
from fitkv import compress_kv_cache

# keys, values, queries: one [H, n, d] tensor per layer, after the prompt pass
budget, lam, kept = compress_kv_cache(keys, values, queries, minimum=128)
```

`keys` and `values` are replaced in place with the compressed tensors. `budget` is the per input budget `B_x` and `lam` is the routing scalar.

A complete example with a Hugging Face model:

```
python examples/compress_prompt.py --model Qwen/Qwen2.5-0.5B-Instruct --budget 128
```

Tests

```
python -m pytest tests/
```

The tests check the budget formula, the routing scalar, the selection rule, that the observation window is always kept, and that every baseline runs and keeps the expected blocks.

Parameters

Name	Default	Meaning
`minimum` (B)	required	Minimum budget in tokens per layer
`block` (P)	16	Tokens per block
`window` (w)	64	Observation window size in queries
`eta`	0.3	Weight of the value term

The scale of `1/2` in `lambda` was fixed by hand and not tuned.

Scope

This repository contains the method and the baselines. It does not include the benchmark harness used for the paper's experiments (dataset loading for LongBench, batch scheduling and result aggregation).

Citation

```
@inproceedings{fitkv2026,
  title={FitKV: Fitting the KV Cache Budget to Each Input for Long Context Inference},
  author={Paul Trust},
  year={2026}
}
```

License

MIT. See LICENSE.